机器学习LAB3报告

522031910213 朱涵

1 DocVQA

1.1 baseline

直接在给出的baseline代码上进行评估，最终准确率为**0.88**。查看错误的问题，发现基本都是因为答案不够完善（比如生成58.5 lbs，而答案只有58.5）、格式不对（比如生成February 24, 1966，而答案却只有其他的时间格式）甚至是少一个空格导致的，真正由于理解或识别导致的错误只有个别。

1.2 提示词优化

在输入问题和图片的同时，给出合适的提示词，尝试获得更高的准确率。prompt如下：

```
prompt = f"""
You are an expert in Document Visual Question Answering (DocVQA).
Your task is to answer questions based on the given document image.

Important Guidelines:
- Focus on textual content in the document
- Pay attention to document structure (tables, forms, etc.)
- For numerical answers, provide the exact value as shown
- For date questions, use the format in the document

Question: {example['question']}

Examples:
Q: What is the invoice number?
A: INV-2023-0042

Q: What is the total amount due?
A: $1,245.00

Now answer this question (only the answer, no other words):
{example['question']}
"""
```

最终准确率仍为**0.88**。查看错误的示例，发现和baseline基本一致。由于上文提到过，baseline本身已经具有高准确率，大多数错误其实是格式或标准答案的问题，因此提示词优化很难使得准确率进一步提高。唯一的方法可能只有针对个别错误问题进行提示词优化（比如显式的告诉VLM需要以什么格式返回时间答案）。个人认为这样的优化不具有实际意义，因此在此不进行实施。

1.3 图像预处理

通过对比度增强、锐化处理、二值化处理，尝试让输入的图片更加清晰，便于VLM提取特征。代码如下：

```python
def preprocess_image(example):
    # ==================== 图像预处理优化 ====================
    # 优化策略:
    # 1. 对比度增强（针对扫描文档）
    # 2. 锐化处理（提高文字清晰度）
    # 3. 二值化处理（减少颜色干扰）
    image = example["image"].convert("RGB")

    # 对比度增强
    enhancer = ImageEnhance.Contrast(image)
    image = enhancer.enhance(1.5)  # 增强50%对比度

    # 锐化处理
    image = image.filter(ImageFilter.SHARPEN)

    # 转换为灰度图（简化颜色信息）
    image = image.convert("L")

    # 二值化处理（可选，根据文档质量）
    # image = image.point(lambda x: 0 if x < 200 else 255, '1')

    return image
    # ==================== END 图像预处理 ====================
```

最终准确率为**0.91**，有所提升。查看结果，可以发现之前个别由于图片不清晰导致识别错误的问题均得到了正确解答，说明图片预处理可以有效的提升VLM的性能。

1.4 上下文

通过上下文学习策略，提供给VLM少样本提示以提高准确率，代码如下：

```python
# 构建few-shot示例文本
few_shot_text = "\n".join([
    f"Example {i+1}:\nQuestion: {ex['question']}\nAnswer: {random.choice(ex['answers'])}"
    for i, ex in enumerate(few_shot_examples)
])

prompt = f"""
You are a document question answering expert. Below are some examples:

{few_shot_text}

Now answer this new question based on the provided document image (only the answer, no other words):
Question: {example['question']}

Important:
- Be precise and extract the exact information
- For numerical answers, don't perform calculations
- If unsure, provide the most likely answer
"""
```

最终准确率为0.84，反而较baseline有所下降。查看结果发现，有一些本来回答正确的问题，由于提供上下文后反而回答的答案比较繁杂，导致不符合标准答案。

2 MP-DocVQA

2.1 baseline

直接在给出的baseline代码上进行评估，只给出第一张图片的准确率为**0.42**，只给出正确图片的准确率为**0.93**。

2.2 图像拼接

通过拼接文档的所有图像提供给VLM，尝试提高准确率。部分代码如下：

```python
# Resize all images to same height while maintaining aspect ratio
resized_images = []
for img in selected_images:
    original_width, original_height = img.size
    if original_height == 0:  # prevent division by zero
        continue

    # Calculate new dimensions
    new_height = min(max_height, original_height)
    new_width = int(original_width * (new_height / original_height))

    # Resize using high-quality downsampling
    resized_img = img.resize((new_width, new_height), Image.LANCZOS)
    resized_images.append(resized_img)

# Calculate total width and max height after resizing
total_width = sum(img.width for img in resized_images)
max_resized_height = max(img.height for img in resized_images) if resized_images else 0
```

最终准确率为**0.47**，虽然比baseline提升了一些，但是不多。推测是因为图像大小限制（20x420x420）导致每一张图片都需要resize，图片的分辨率变小导致模型更难提取特征，从而准确率并没有提升很多。如果不限制图像大小，或许可以获得更高的准确率。

2.3 OCR+RAG

通过OCR+RAG尝试提高模型准确率，具体而言，先用OCR识别提取出每一张图片的文本，然后对这些文本和问题进行RAG检索，其中向量嵌入采用BERT模型，代码如下：

```python
# Iterate through all available images (assuming up to 5 images)
for i in range(1, 6):
    image_key = f"image_{i}"
    if image_key not in example:
        continue

    image = example[image_key]

    # Extract text from image
    image_text = get_image_text(image)

    if not image_text.strip():  # Skip if no text found
        continue

    # Get text embedding
    text_embedding = get_text_embedding(image_text)

    # Calculate cosine similarity
    similarity = np.dot(question_embedding, text_embedding) / (
        np.linalg.norm(question_embedding) * np.linalg.norm(text_embedding)
    )

    # Update best image if current is better
    if similarity > best_similarity:
        best_similarity = similarity
        best_image = image
```

最终准确率为**0.64**，可以看到基于OCR和RAG，模型对于图片和问题的理解更加准确了。

2.4 图片增强

尝试通过图片增强提高准确率，基于只提供正确图片的策略。图片预处理的代码和第一部分的类似：

```python
def preprocess_image(example):
    """改进部分：图像增强+OCR文本提示
    目的：通过图像增强提高可读性，并提取关键文本作为提示"""

    idx = int(example["answer_page_idx"])+1
    image = example[f"image_{idx}"]

    # 图像增强处理
    # 1. 对比度增强
    enhancer = ImageEnhance.Contrast(image)
    image = enhancer.enhance(1.5)
    # 2. 锐化
    image = image.filter(ImageFilter.SHARPEN)
    # 3. 二值化（适用于文档）
    image = image.convert('L').point(lambda x: 0 if x < 128 else 255, '1')

    return image
```

最终准确率为**0.95**，可以看到图片预处理对于模型的性能有微小提升。